

Blink

Réseau social multimédia boosté à l'IA

Rapport technique — Projet Django 2026

Poste. Parle. L'IA fait le reste.

Équipe

Chouat Ayoub · Kardamine Jasser · Ben Taleb Ilef · Azizi Amine

28 octobre 2026

1. Présentation du projet

Blink est une application web **Django 5.2** qui permet de publier des photos accompagnées de messages vocaux. L'intelligence artificielle écrit des légendes stylisées, propose des hashtags, transcrit les vocaux, crée des images artistiques et reconnaît les amis sur les photos. Les huit fonctionnalités prévues dans la présentation sont livrées et fonctionnent même sans clé d'API, grâce à des moteurs locaux de repli.

Fonctionnalité	Ce que voit l'utilisateur	Technologie	Sans clé d'API
Publication multimédia	Photo + message vocal enregistré dans le navigateur	Django, Pillow, MediaRecorder (JS)	Oui
Légendes stylisées	Poétique, marketing ou humoristique en un clic	Gemini	Générateur local
Hashtags automatiques	5 à 8 hashtags à sélectionner	Gemini	Générateur local
Images IA	Filtres artistiques, images dérivées, avatars	OpenCV, Pollinations.ai, Hugging Face (option payante)	Oui
Transcription audio	Texte sous chaque vocal (accessibilité)	Gemini, puis Whisper	Saisie manuelle
Reconnaissance faciale	« 2 amis reconnus » puis confirmation des tags	OpenCV YuNet + SFace	Oui (100 % local)
Système social	Réactions, commentaires texte et vocaux, amis, abonnements, notifications	Django ORM	Oui
Authentification	Inscription, connexion par e-mail ou pseudo, profils	Django Auth	Oui



Page d'accueil publique (charte graphique : Maya Blue, Midnight Violet, Sora / Inter)

2. Architecture

Blink suit le pattern **MVT** (Modèle – Vue – Template) de Django. Le navigateur appelle des vues génériques (pages HTML) ou l'API REST (JSON). Les deux passent par les modèles et l'ORM. Les appels d'IA lents (transcription, détection de visages, génération d'images) sont lancés dans des tâches d'arrière-plan après l'enregistrement : la page répond tout de suite et affiche le résultat dès qu'il est prêt.

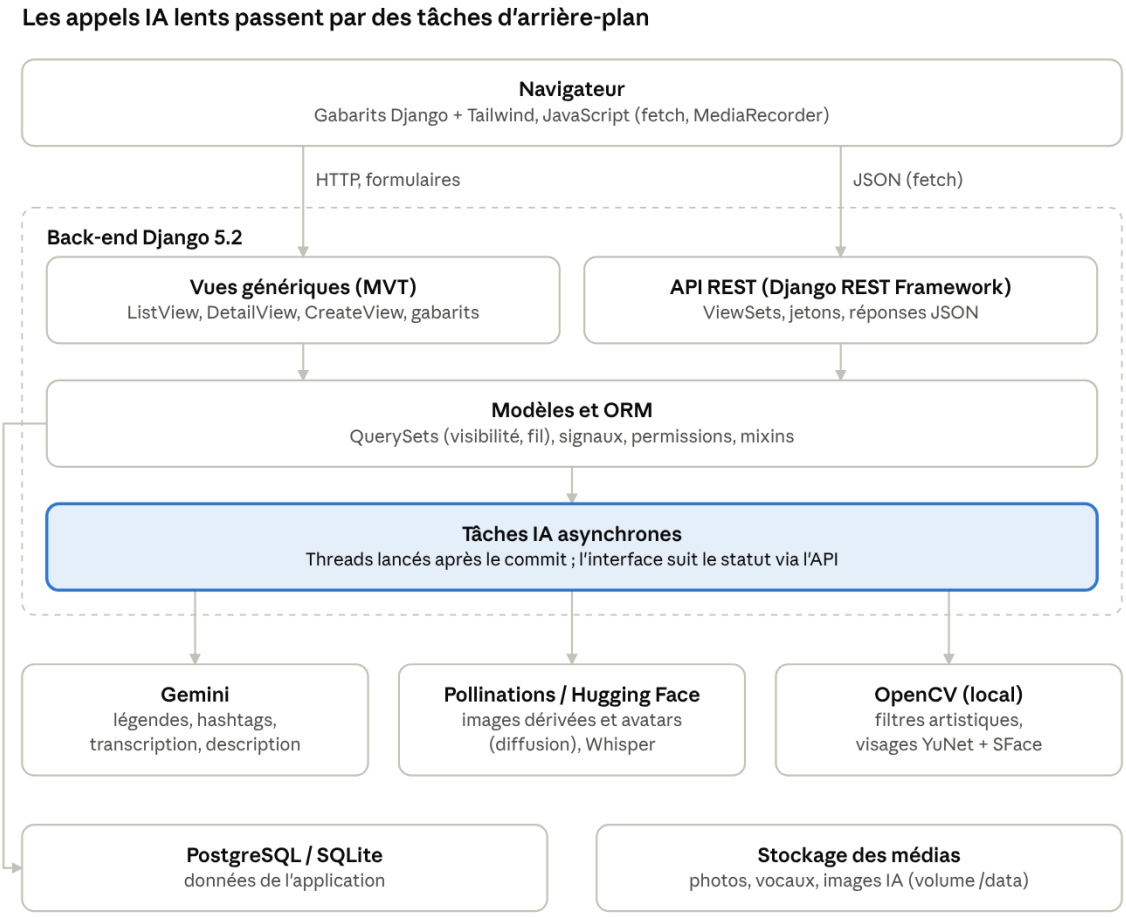


Figure 1 — Architecture de Blink en quatre couches

L'interface utilise les gabarits Django plutôt que React, car la grille d'évaluation note le moteur de gabarits et les classes génériques. L'API REST couvre toute l'application et permet d'ajouter plus tard le front-end React prévu dans la présentation, ou une application mobile.

3. Structure du projet et modules

Le projet compte sept applications Django : une par « gestion » du cahier des charges, plus le noyau commun et l'API.

Application	Gestion	Modèles	Vues génériques	IA / bibliothèques
`accounts`	Commune : utilisateurs, profils, amis	User, Profile, Friendship, Follow	CreateView, DetailView, UpdateView, TemplateView, LoginView	Django Auth
`posts`	1 — Publications	Post, Reaction, Comment, AIGeneration	ListView, DetailView, CreateView, UpdateView, DeleteView	Gemini, Pillow
`audio`	2 — Audio & transcription	AudioPost, Transcription, AudioComment	ListView, CreateView, DetailView, UpdateView, DeleteView	Gemini, Whisper
`aistudio`	3 — IA générative d'images	GeneratedImage, AIPromptTemplate, ImageFilter	ListView, CreateView, DetailView, UpdateView, DeleteView	OpenCV, Pollinations, Hugging Face
`faces`	4 — Reconnaissance faciale	FaceEncoding, DetectedFace, TagSuggestion	DetailView, TemplateView	OpenCV YuNet + SFace
`core`	Accueil, recherche, modération	Notification, Report	ListView, TemplateView	Admin personnalisé, balises
`api`	API REST	—	ModelViewSet, ReadOnlyModelViewSet	Django REST Framework

Arborescence principale :

```
blink/ configuration : settings.py, urls.py, wsgi.py
accounts/ posts/ audio/ aistudio/ faces/ core/ api/
models.py views.py urls.py forms.py admin.py tests.py
templates/ base.html, partials/, une page par vue, admin/
static/ css/app.css (Tailwind compilé), js/blink.js, img/
core/templatetags/blink_tags.py balises et filtres personnalisés
Dockerfile, docker-compose.yml, docker/entrypoint.sh déploiement
```

Les modèles reprennent les entités du document des gestions et du diagramme de classes. Le *Like* du diagramme devient *Reaction* (quatre types) et *Media* devient *AudioPost*.

4. Module Comptes & amis (accounts)

Ce module est la base dont dépendent les quatre gestions : il porte l'utilisateur personnalisé, son profil et le graphe social.

- **User** hérite d'`AbstractUser` : e-mail unique, photo de profil (référence pour la reconnaissance faciale), bio. Méthodes du diagramme de classes : `follow()`, `unfollow()`, `is_following()`, plus `friends()` et `is_friend()`.
- **Profile** (OneToOne) : avatar, date de naissance, ville ; créé automatiquement par un signal `post_save`.
- **Friendship** : demande d'ami avec statut (en attente, acceptée, refusée), contraintes d'unicité et interdiction de s'ajouter soi-même.
- **Follow** : abonnement asymétrique, modèle intermédiaire d'un ManyToMany.

- **Vues** : `SignUpView` (CreateView), `BlinkLoginView` (LoginView), `ProfileDetailView`, `ProfileUpdateView` (deux formulaires dans une UpdateView), `FriendsView`, et des vues POST pour demander, accepter, refuser ou retirer un ami et s'abonner.
- **Authentification** : `EmailOrUsernameBackend` permet de se connecter avec l'e-mail ou le pseudo.

5. Module Publications (posts)

C'est le cœur du réseau social : une publication photo, ses réactions, ses commentaires et la légende générée par Gemini. Les trois autres gestions dépendent de ce module.

Modèles

Modèle	Champs principaux	Rôle
Post	auteur, image, caption, caption_ia, hashtags_ia, caption_style, visibilite, est_masque, dates	La publication ; méthodes `generate_ai(style)`, `get_hashtags()`, `get_likes()`, `tagged_users()`
Reaction	user, post, type (like, love, haha, sad)	Une réaction par utilisateur et par post ; `Reaction.toggle()`
Comment	user, post, contenu, date	Commentaire texte (les vocaux sont dans le module audio)
AIGeneration	post, user, type, moteur, prompt, result (JSON)	Journal de chaque appel d'IA, affiché dans l'admin

Règles de visibilité (PostQuerySet)

- `visible_to(user)` : un visiteur voit les posts publics ; un membre voit aussi les siens et les posts « amis » de ses amis ; un post « privé » n'est visible que par son auteur ; un post masqué par la modération n'apparaît nulle part.
- `feed_for(user)` : le fil contient mes posts, ceux de mes amis et ceux des comptes que je suis, par ordre chronologique (pas d'algorithme).
- `with_stats(user)` : ajoute le nombre de réactions et de commentaires, ma propre réaction, et précharge les données liées pour éviter les requêtes en boucle.

Vues et URL

URL	Vue (classe générique)	Action
/posts/nouveau/	PostCreateView (CreateView)	Publier une photo, un vocal optionnel et la légende IA
/posts/<id>/	PostDetailView (DetailView)	Voir le post, ses réactions et ses commentaires
/posts/<id>/modifier/	PostUpdateView (UpdateView)	Modifier la légende, les hashtags, la visibilité
/posts/<id>/supprimer/	PostDeleteView (DeleteView)	Supprimer (auteur uniquement)
/posts/<id>/reagir/	ReactView (View)	Réagir sans recharger la page (réponse JSON)
/posts/<id>/commenter/	CommentCreateView (CreateView)	Commenter
/posts/<id>/signaler/	ReportCreateView (CreateView)	Signaler à la modération

URL	Vue (classe générique)	Action
/posts/explorer/, /posts/tag/<tag>/	ExploreView, HashtagView (ListView)	Explorer, filtrer par hashtag

Génération de légendes (Gemini)

La photo est réduite à 1 024 px puis envoyée à Gemini avec le style choisi ; la réponse JSON contient une légende et 5 à 8 hashtags, nettoyés (minuscules, sans accents). Sans clé, un générateur local (`posts/ai.py`) déduit une ambiance des couleurs de l'image (coucher de soleil, mer, nature, nuit, pastel) et remplit des modèles de phrases propres à chaque style. À la publication, un vocal optionnel crée un AudioPost ; la transcription et la détection de visages partent en tâche d'arrière-plan.

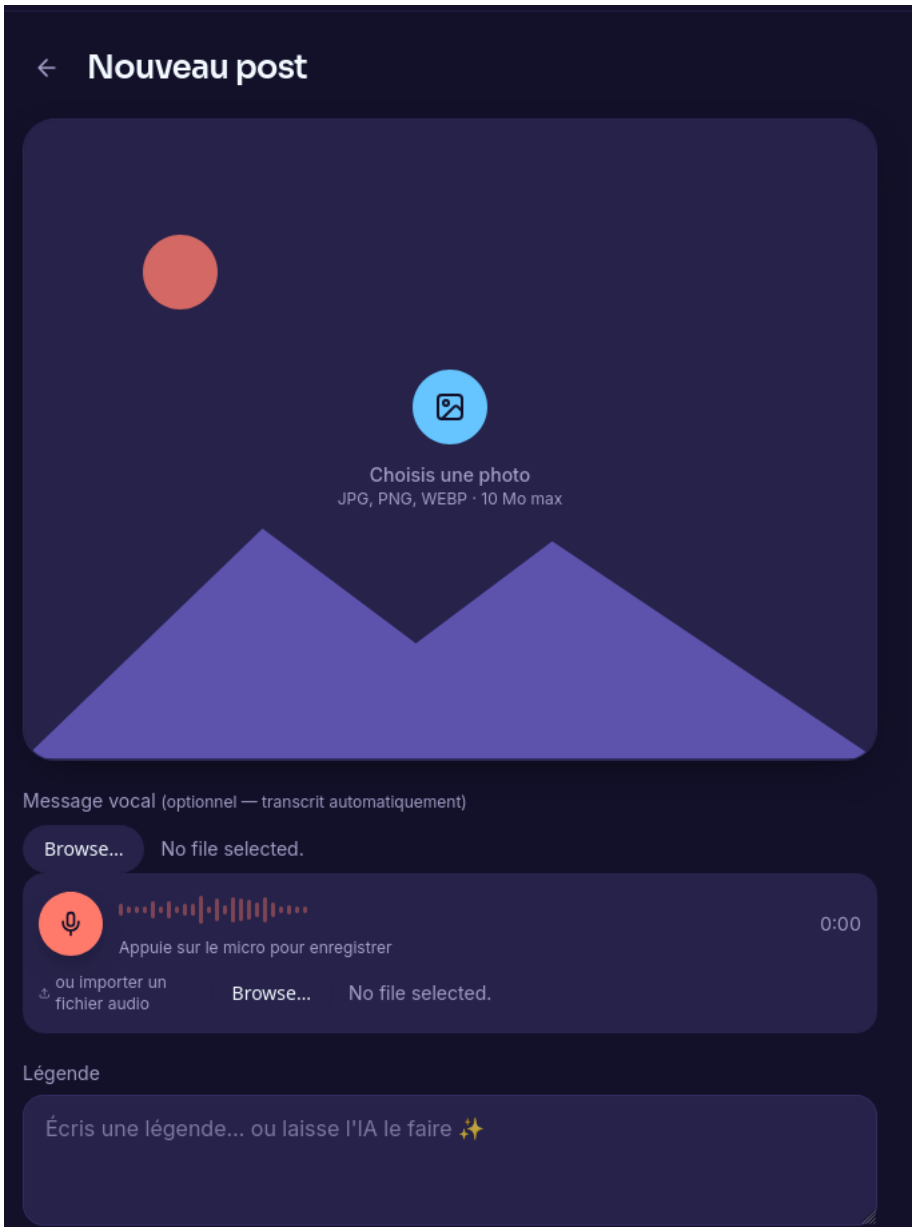


Figure 2 — Écran « Nouveau post » : photo, message vocal, légende et panneau « Résultat IA »

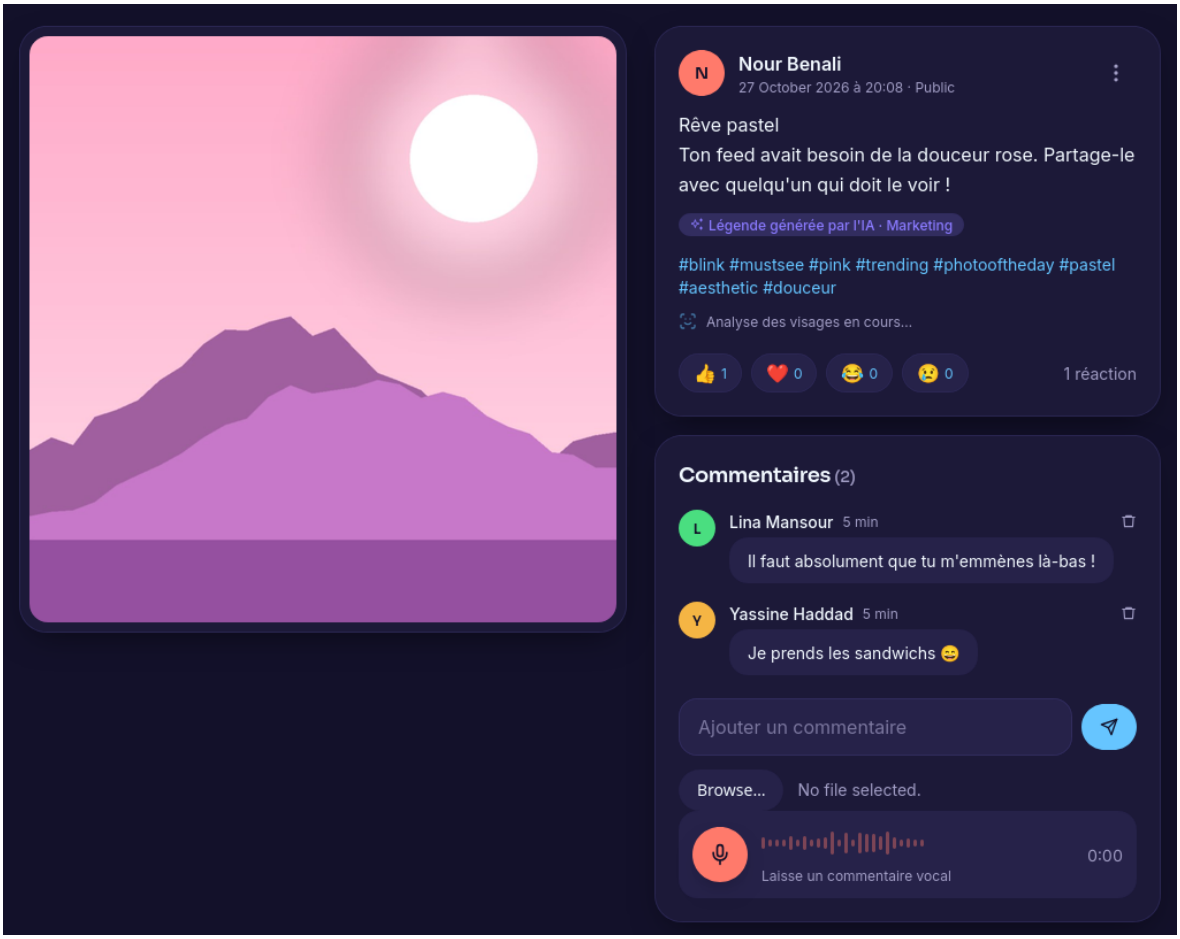


Figure 3 — Détail d'une publication : légende IA, hashtags, réactions et commentaires

6. Module Audio & transcription (audio)

Les utilisateurs publient des vocaux seuls, attachés à une photo ou en commentaire, et l'IA les transcrit pour les personnes malentendantes.

- **AudioPost** : user, post (optionnel), fichier audio, description, durée.
- **Transcription** : texte, langue, statut (en attente, terminée, erreur), moteur, message d'erreur.
- **AudioComment** : commentaire vocal sur un post, lié à sa transcription.
- **Vues** : `AudioPostListView`, `AudioPostCreateView`, `AudioPostDetailView`, `AudioPostDeleteView`, `TranscriptionUpdateView` (correction manuelle), `RetranscribeView`, `AudioCommentCreateView`.
- **Enregistrement** : API MediaRecorder du navigateur (5 min max) ou import de fichier ; formats webm, ogg, mp3, wav, m4a, flac ; 15 Mo maximum.
- **IA** : Gemini transcrit d'abord (palier gratuit), puis Whisper via Hugging Face en secours. La langue est détectée. Sans service, le statut passe à « erreur » et l'auteur peut saisir le texte.

7. Module Studio IA (aistudio)

Le studio transforme les photos de l'utilisateur de trois façons : filtre artistique, image dérivée par diffusion et avatar.

Type	Comment l'image est produite	Modèle	Coût
Filtre IA	Stylisation fidèle de la photo : anime, peinture, cyberpunk, aquarelle, croquis, vintage	OpenCV + Pillow	Gratuit, local
Image dérivée	Gemini décrit la photo, puis un modèle de diffusion la redessine dans le style choisi	Pollinations.ai (flux)	Gratuit, sans clé
Avatar	Même principe à partir de la photo de profil	Pollinations.ai ou Stable Diffusion 3 (Hugging Face)	Hugging Face : payant

- **GeneratedImage** : post (ou aucun pour un avatar), type, image originale, image générée, prompt, style, modèle, statut.
- **AIPromptTemplate** : modèles de prompt personnels avec les variables `{description}`, `{style}`, `{caption}`.
- **ImageFilter** : six filtres système créés par une migration de données, plus les filtres de l'utilisateur (style de base, intensité, saturation, contraste).
- **Vues** : galerie, génération depuis un post, avatar, « utiliser comme avatar », « publier », et le CRUD complet des filtres et des modèles de prompt.

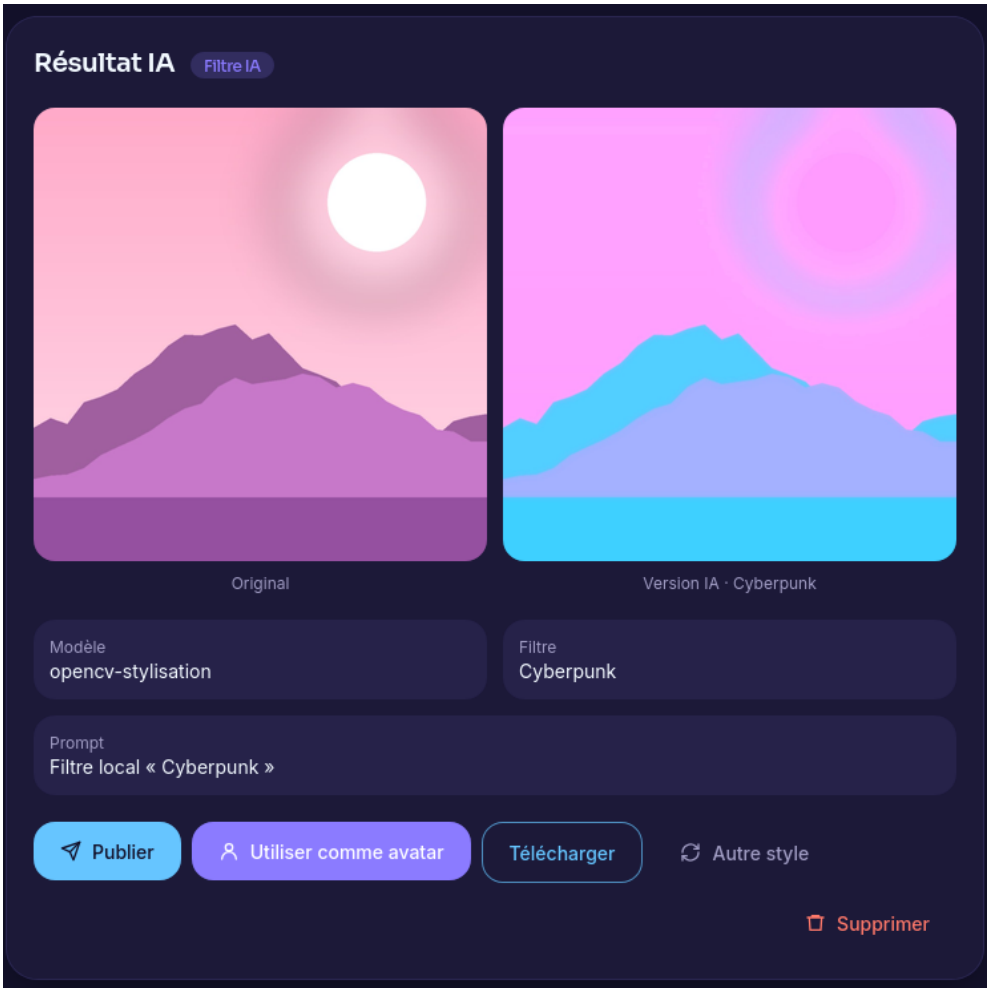


Figure 4 — Résultat d'un filtre « Cyberpunk » dans le Studio IA

8. Module Reconnaissance faciale (faces)

Quand un utilisateur publie une photo de groupe, Blink détecte les visages, les compare aux empreintes de ses amis et propose des tags à confirmer. Tout tourne en local, sans service externe.

- **FaceEncoding** : vecteur de 128 valeurs calculé depuis la photo de profil.
- **DetectedFace** : visage trouvé sur un post (cadre en pourcentage, visage recadré, ami identifié, confiance).
- **TagSuggestion** : suggestion IA ou manuelle, statut en attente, accepté ou refusé.
- **Moteur** : modèles ONNX d'OpenCV Zoo — YuNet (détection, 227 Ko) et SFace (empreinte, 37 Mo), téléchargés par `python manage.py setup_face_models`. Un ami est reconnu au-delà d'une similarité cosinus de 0,363.
- **Règles** : seuls les amis de l'auteur sont proposés ; l'auteur confirme ; la personne taguée est notifiée et peut retirer le tag ; chacun peut supprimer son empreinte faciale.
- **Résultat mesuré** : sur une photo de test avec deux visages, les deux ont été attribués à l'ami attendu avec 82 % et 91 % de confiance.

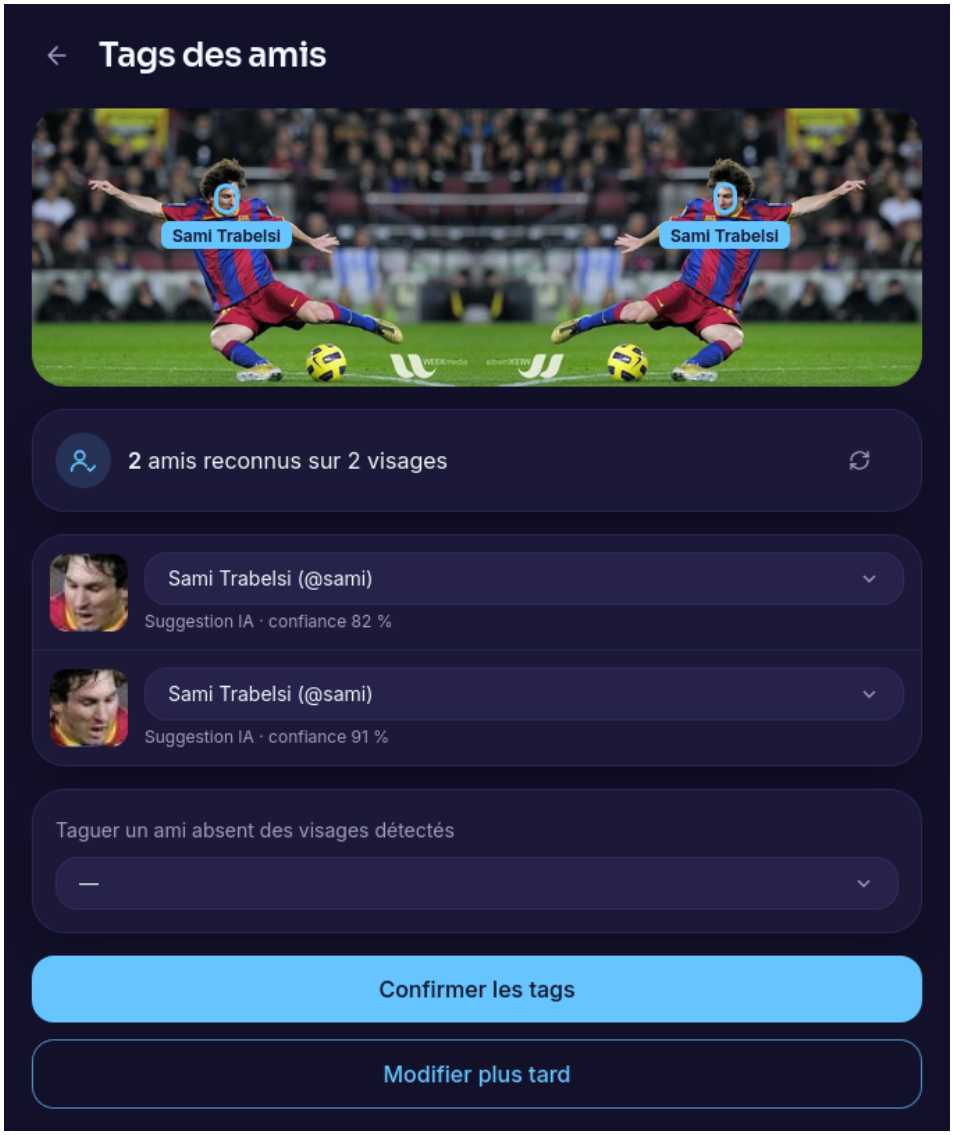


Figure 5 — Écran « Tags des amis » : visages détectés et suggestions

9. Noyau et API REST (core, api)

- **Notification** (réaction, commentaire, demande d'ami, tag, traitement IA…) et **Report** (signalement : spam, haine, contenu inapproprié, harcèlement).
- **Vues** : `HomeView` (accueil pour les visiteurs, fil pour les membres), `SearchView`, `NotificationListView`.
- **Tâches asynchrones** (`core/tasks.py`) : les traitements IA lents tournent dans un thread lancé après le commit de la base.
- **Permissions** : `OwnerRequiredMixin` (LoginRequiredMixin + UserPassesTestMixin) réserve modification et suppression au propriétaire.
- **Gabarits** : héritage depuis `base.html`, gabarits partiels réutilisables, balises personnalisées `{% icon %}`, `{% avatar %}`, `{% waveform %}` et filtres `hashtagify`, `duration`, `short_since`.

Principales routes de l'API (Django REST Framework, authentification par session ou par jeton)

Méthode	URL	Rôle
POST	/api/auth/token/	Obtenir un jeton d'authentification
GET, POST	/api/posts/ (?feed=1, ?auteur=, ?hashtag=)	Fil et création de publication
POST	/api/posts/<id>/react/	Réagir
GET, POST	/api/posts/<id>/comments/	Lire et écrire des commentaires
POST	/api/ai/caption/	Légende + hashtags IA avant publication
GET	/api/posts/<id>/faces/	Visages détectés et suggestions
CRUD	/api/audio-posts/, /api/transcriptions/	Audio
CRUD	/api/generated-images/, /api/filters/	Studio IA
GET	/api/users/, /api/notifications/, /api/ai/status/	Social et état des services IA

10. Administration

L'administration, accessible sur `/admin/` aux comptes membres du personnel, combine un tableau de bord Blink fait sur mesure et l'interface Django pour gérer chaque modèle. Elle couvre les trois cas d'utilisation de l'acteur Administrateur : modérer les publications, gérer les utilisateurs et consulter les signalements.

Comment le tableau de bord est branché

1. `core/admin_config.py` déclare `BlinkAdminConfig`, qui remplace `django.contrib.admin` dans INSTALLED_APPS et pointe vers un site personnalisé.
2. `core/admin_site.py` définit `BlinkAdminSite` (sous-classe d'AdminSite) : titre « Blink · Administration » et méthode `index()` qui ajoute les statistiques.
3. `templates/admin/blink_index.html` affiche les statistiques ; `templates/admin/base_site.html` applique la charte Blink.
4. Chaque application a son `admin.py` : les `@admin.register` s'enregistrent sur ce site.

Contenu du tableau de bord

Bloc	Ce qu'il montre
Indicateurs clés	Utilisateurs (+ inscrits de la semaine), publications (+ masquées), interactions, amitiés (+ demandes en attente)
Histogramme	Nombre de publications par jour sur les 14 derniers jours
Intelligence artificielle	Légendes IA, transcriptions, images IA, visages détectés, tags confirmés, vocaux, utilisation par moteur
Signalements à traiter	Les 8 derniers signalements non traités, avec miniature et lien « Traiter »
Alertes	Nombre de transcriptions en erreur

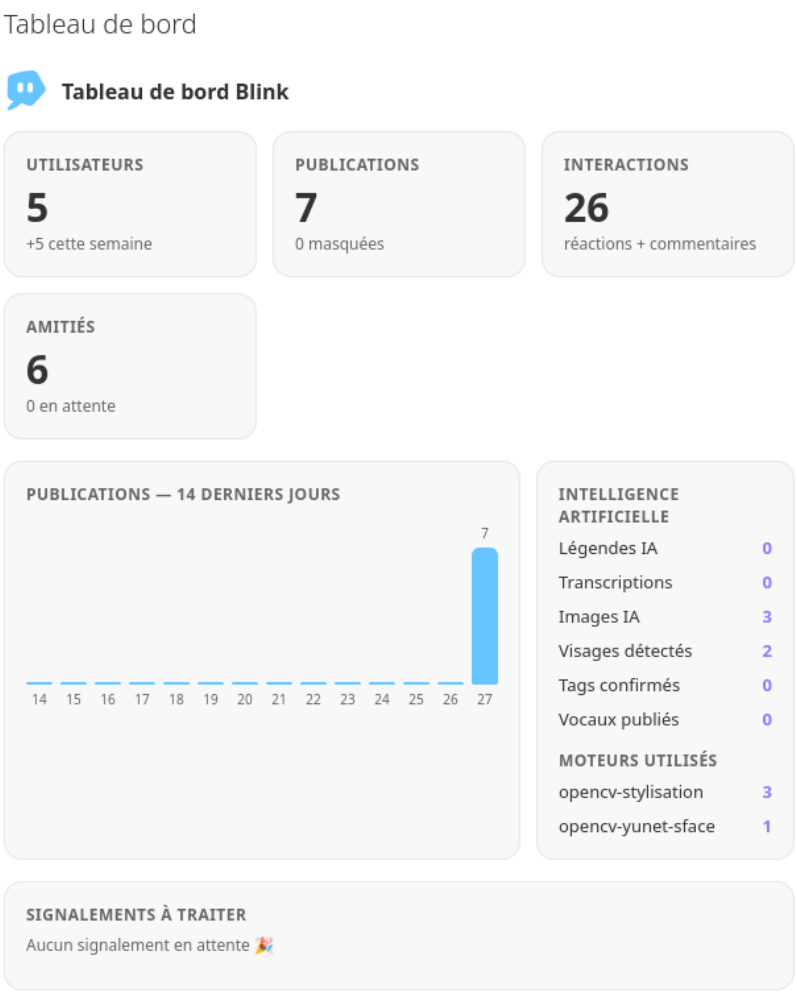


Figure 6 — Tableau de bord de l'administration

Écrans de gestion et actions

Modèle	Affichage et filtres	Actions de l'administrateur
Utilisateurs	Avatar, pseudo, e-mail, nombre de posts, statut ; recherche ; profil en ligne	Désactiver ou réactiver des comptes
Publications	Miniature, légende, visibilité, réactions, signalements en rouge ; filtres ; aperçu ; vocaux, commentaires et signalements en ligne	Masquer (et clore les signalements), rendre visible
Signalements	Miniature du post, motif, auteurs ; case « traité » modifiable dans la liste	Masquer les publications signalées, classer sans suite
Audio et transcriptions	Lecteur audio intégré, statut, langue, moteur, erreur	Relancer la transcription IA
Images générées, filtres, prompts	Miniatures avant / après, style, modèle, statut	Gestion complète
Visages, encodages, tags	Visage recadré, ami identifié, confiance	Gestion complète
Générations IA	Type, moteur, prompt, résultat JSON	Lecture seule (journal d'audit)

Une publication masquée disparaît immédiatement du fil, de l'exploration, de la recherche et de l'API, car `visible_to()` exclut `est_masque=True`.

11. Construire et lancer le projet

Prérequis : Python 3.12. Docker pour le déploiement. Node.js seulement pour recompiler le CSS.

En local (développement, base SQLite)

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
cp .env.example .env # ajouter GEMINI_API_KEY (optionnel)
.venv/bin/python manage.py migrate
.venv/bin/python manage.py setup_face_models
.venv/bin/python manage.py seed_demo --admin
.venv/bin/python manage.py runserver # http://127.0.0.1:8000
```

Comptes de démonstration : nour, sami, yassine, lina (mot de passe blink12345) et admin / admin12345.

Avec Docker (production, PostgreSQL + Gunicorn)

```
docker compose up -d --build # http://127.0.0.1:8000
docker compose down # arrêter (-v pour effacer les données)
```

Variable	Rôle	Défaut
SECRET_KEY	Clé secrète Django (à changer en production)	clé de développement
DEBUG	Mode débogage	1 en local, 0 dans Docker
DATABASE_URL	Base de données	SQLite (PostgreSQL dans Docker)
GEMINI_API_KEY	Légendes, hashtags, transcription	vide : mode local
IMAGE_PROVIDER	pollinations, huggingface ou local	pollinations
HF_TOKEN	Hugging Face (crédits payants)	vide
SEED_DEMO	Données de démo au démarrage du conteneur	1

12. Tests et grille d'évaluation

Les 67 tests automatisés passent (`python manage.py test`). Ils simulent les services d'IA externes et testent la reconnaissance faciale sur une vraie photo.

Critère de la grille	Où le trouver dans Blink
Décrire l'architecture du Framework	Pattern MVT, section 2, blink/settings.py et urls.py
Reconnaître les composants et la structure	Sept applications, une par gestion (section 3)
Utiliser les classes génériques	ListView, DetailView, CreateView, UpdateView, DeleteView, TemplateView, mixins ; ViewSets
Mettre en place le Dashboard admin	BlinkAdminSite, statistiques, actions de modération (section 10)
Concevoir le Back End	Modèles, QuerySets, signaux, permissions, tâches asynchrones
Manipuler le moteur de templating	Héritage, inclusions, balises et filtres personnalisés
Comprendre le langage Python	Services IA, traitement d'images NumPy / OpenCV, commandes de gestion
Construire des classes et méthodes	User.follow(), Post.generate_ai(), Reaction.toggle(), FaceEngine
Construire des APIs	Django REST Framework, jetons, 11 ressources
Déploiement d'une application web	Dockerfile, docker-compose (PostgreSQL + Gunicorn + WhiteNoise)